

Cahier des charges du projet de Fin d'études : Dossier Médical Sécurisé avec Blockchain

Contexte et Enjeux :

La gestion des données médicales représente un défi majeur dans le domaine de la santé. Les systèmes traditionnels souffrent de vulnérabilités face aux cyberattaques, aux erreurs humaines et aux accès non autorisés. Pour garantir la confidentialité, l'intégrité et la transparence des informations sensibles, la technologie **Blockchain** s'impose comme une solution innovante et efficace.

Notre projet, "**Dossier Médical avec Blockchain**", vise à développer une application décentralisée (**DApp**) permettant de stocker et sécuriser les dossiers médicaux des patients à l'aide de **smart contracts** sur la blockchain **Ethereum**.

L'application s'appuiera sur une interface graphique performante en **Python (PyQt/Tkinter)**, garantissant une expérience utilisateur fluide et intuitive.

Objectifs du Projet :

- **Sécurisation des données médicales** : Assurer l'immuabilité et la confidentialité des dossiers médicaux via la blockchain.
- **Confidentialité** : Implémenter des mécanismes avancés de cryptage pour protéger les informations médicales.
- **Transparence et traçabilité** : Offrir aux utilisateurs un suivi détaillé de toutes les interactions avec les dossiers médicaux.
- **Décentralisation** : Éliminer la dépendance à un système centralisé pour une autonomie renforcée.
- **Interopérabilité** : Permettre un partage sécurisé des données entre différents professionnels de santé.

Technologies Employées :

- **Blockchain Ethereum** : Stockage des transactions et gestion des smart contracts.

- **Solidity** : Développement des smart contracts assurant l'administration des dossiers médicaux et des autorisations d'accès.
- **IPFS (InterPlanetary File System)** : Hébergement des fichiers médicaux hors blockchain, avec enregistrement des hachages pour garantir leur authenticité.
- **Frontend (Python):**
 - **Python (PyQt/Tkinter)** : Expérience utilisateur intuitive et simplifiée.
- **Interaction avec la Blockchain :**
 - **Python** : Intégration via **Web3.py** pour interagir avec Ethereum et les smart contracts déployés.

Fonctionnalités :

- **Enregistrement des Patients** : Chaque patient possède une adresse unique sur la blockchain, liée à ses informations de base qu'elles sont enregistrées via un smart contract sécurisé.
- **Création et Mise à Jour des Dossiers Médicaux** : Stockage sécurisé des dossiers sur IPFS, avec un hachage conservé sur la blockchain.
- **Gestion des Accès et Partage Sécurisé** : Les patients peuvent autoriser et révoquer l'accès à leurs dossiers médicaux à différents professionnels de santé. Les permissions sont gérées via les smart contracts.
- **Traçabilité et Historique** : Toutes les actions effectuées sur les dossiers médicaux sont enregistrées sur la blockchain, offrant une traçabilité complète des modifications et consultations.
- **Cryptage Avancé** : Les données sensibles sont cryptées avant d'être envoyées à IPFS. Seules les personnes autorisées peuvent déchiffrer et consulter ces informations .
- **Authentification Décentralisée** : Identification des utilisateurs (patients et médecins) via **portefeuilles Ethereum** (MetaMask) ou clés cryptographiques privées.

Architecture du Système :

- **Frontend** : Interface utilisateur en **Python (PyQt/Tkinter)**, avec interaction blockchain via **Web3.py**.
- **Backend** : Smart contracts en **Solidity**, déployés sur **Ethereum**.
- **Stockage Décentralisé** : Les fichiers médicaux sont hébergés sur **IPFS**, seuls leurs hachages sont stockés sur la blockchain.
- **Smart Contracts** :
 - **ContractPatient** : Gestion des enregistrements des patients.

- **ContractDoctor** : Administration des permissions et accès aux dossiers médicaux.
- **ContractAudit** : Journalisation des actions pour assurer une traçabilité complète.

Déroulement du Projet :

Étape 1 : Conception & Planification

- Définir les besoins techniques et concevoir l'architecture de l'application.
- Conception de l'architecture globale et des interactions avec Ethereum via Web.py.

Étape 2 : Développement

- Création du **frontend** en **Python (PyQt/Tkinter)**.
- Développement des **smart contracts** en Solidity pour la gestion des dossiers et des autorisations.

Étape 3 : Tests & Déploiement

- Validation des smart contracts sur un réseau de test (**Ropsten, Rinkeby**).
- Déploiement final des smart contracts sur le réseau principal Ethereum et des front-ends.

Étape 4 : Documentation & Soutenance

- Rédaction complète de la documentation technique.
- Présentation d'une démonstration de l'application fonctionnelle avec interaction en temps réel.

Informations Complémentaires :

- Travail individuel
- Encadré par : Prof. Souad Amjad
- Date de dépôt & soutenance : à définir